

DSCI-D 532: Applied Database Technologies

D. Scrivner

7/16/26

Wyatt Keller

Final Project Milestone 2

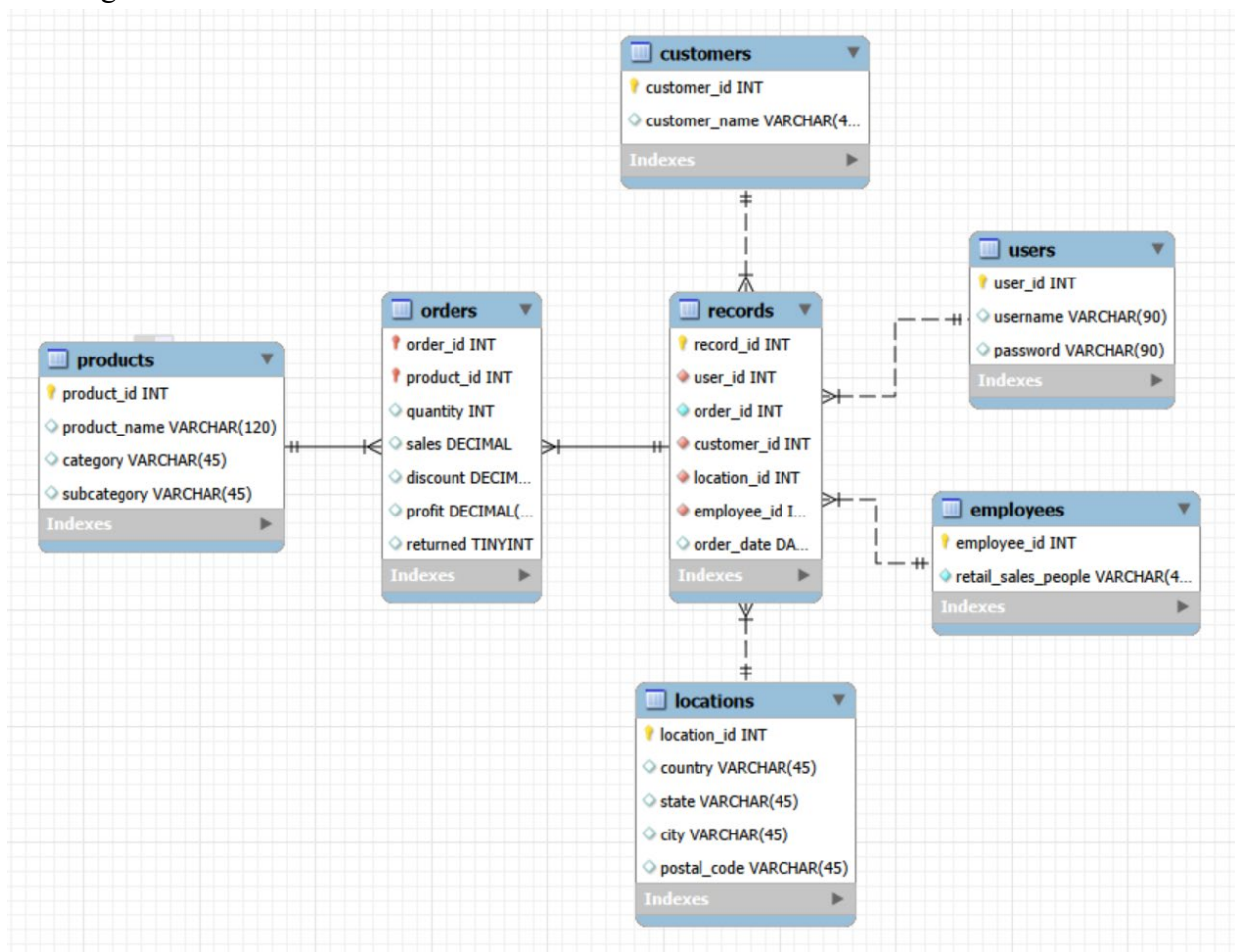
QuickCast: Forecasting Business Intelligence Made Easy

Wyatt Keller

Part 1: Conceptual Diagram and Schema

Below is an ER Diagram dictating the schema and concept of the QuickCast backend database.

ER Diagram:



One important thing to note for transparency is that this diagram was created through MySQL Workbench, and since this database is made through SQLite, not everything transfers fully.

Diagram Structure:

The QuickCast database follows something similar to the classic star schema design. The facts table is records, which have various relationships to customers, orders, employees, and locations. The only other table which has a connection to another table other than records is orders, which has a connection to products. This allows for the database to be reasonably normalized to a 2nd level of normalization. With this normalization level, the backend logic is much clearer than it would be for a higher level. Important to note, this table stores all users data into one database, hence why the users table is present.

Entities and Relationships:

The entities are:

records
locations
employees
users
customers
orders
products

They have the following relationships:

customers -> records (one to many)

users -> records (one to many)

locations -> records (one to many)

employees -> records (one to many)

records -> orders (one to many)

products -> orders (one to many)

Assumptions and Design Decisions:

Due to this being a more lightweight application for smaller businesses, I have decided to go with a single database for all users approach. This is much easier to implement on the backend, with only requiring the simple added step of doing a filtering subquery on the records table based on the user. Whenever a user signs up, they will be assigned a user_id to differentiate their data

from everyone else's. Furthermore, whenever a user uploads data, then providing id's for the data will be optional. If they do provide ids, then they will be appended to the database sequentially while respecting the uniqueness of the id. If no id is provided, it will be assumed that each entity would be unique and assigned their own id. For example, if a user uploaded 2 customers with the same name, but provided different ids for them, it would be respected in the QuickCast database. If no id was provided for those customers, they will be assigned the same id.

Part 2: Database Constraints

Required fields / no null values

The following are required fields:

record_id – is a primary key of the records table, while technically redundant with order_id, it is essential in the case that more tables are added to the database

user_id and username – is the primary key and name of each user within the database. For this database to have a login feature, both fields are required to be present. There will not be an enforcement of a password for each user

location_id, country, city – is the primary key for the locations table and the key information for shipping orders

customer_id, customer_name – is the primary key for the customers table and is necessary information for a transaction to know who the customer in an order was

order_id, product_id, quantity, sales, profit – order_id and product_id create a composite primary key for the orders table, and as such are both required. The quantity, sales, and profit are all essential parts to an order for the business intelligence calculations I want the application to do

product_id, product_name – is the primary key and bare minimum information for a purchase within an order

employee_id, retail_sales_people – is the primary key and name for each employee which is involved within the transaction, though to note, employee_id is not required within the records table

Unique Values and Primary Keys

The following are unique values and primary keys:

records table – record_id, order_id – while having both unique values is redundant, it should allow for easy sub querying according to the user while also allowing for easy joins to the orders table

customers table – customer_id – as the primary key it needs to be unique

users table – user_id – as a primary key it needs to be unique

products table – product_id – as a primary key it needs to be unique

locations table – location_id – as a primary key it needs to be unique

employees table – employee_id – as a primary key it needs to be unique

Composite Key

The following are composite keys:

orders table – order_id, product_id – They combine to make a composite key necessary since each order has multiple products, and so each row has to be differentiated

Foreign Key

The following are foreign keys:

records table – user_id, customer_id, location_id, employee_id – these reference user_id, customer_id, location_id, and employee_id within the users, customers, locations, and employees tables respectively

orders table – order_id, product_id – these reference order_id and product_id within the records and products table respectively

Data type choices

Due to SQLite's typing, the main types are NULL, Text, Integer, Real, and Blob, which I why the ER diagram provided above was not fully accurate. Furthermore, the Real type will not be used often due to it not being precise to do numbers with, and so the workaround will be that

sales stored will be multiplied by 100 and be integers. Here are the types for each fields of the following tables:

records table:

record_id – Integer

user_id – Integer

order_id – Integer

customer_id – Integer

location_id – Integer

employee_id – Integer

order_date – Text

customers table:

customer_id – Integer

customer_name – Text

users table:

user_id – Integer

username – Text

password – Text

employees:

employee_id – Integer

retail_sales_people – Text

locations:

location_id – Integer

country – Text

state – Text

city – Text

postal_code – Integer

orders:

order_id – Integer

product_id – Integer

quantity – Integer

sales – Integer

discount – Integer

profit – Integer

returned – Integer

products:

product_id – Integer

product_name – Text

category – Text

subcategory – Text

Formatting rules:

Dates will be done in the standard format of YYYY-MM-DD

All numbers to deal with sales will be multiplied by 100 in the backend, and divided by 100 when sent to the frontend

Formatting will depend often on the user data they upload with their consistency, as QuickCast aims to be a visualization platform and not correct the user data. Many of the SQLite statements per user will use GROUP BY, and as such shouldn't be affected too much.

For the database queries and creations, please see the python scripts within the GitHub, or the scripts included within this assignment submission.

The GitHub can be found here: [thatguy-jpeg/QuickCast---Forecasting-Done-Quickly](https://github.com/thatguy-jpeg/QuickCast---Forecasting-Done-Quickly)

The data used can be found here: <https://www.kaggle.com/datasets/shandeep777/retail-supply-chain-sales-dataset>

Part 4: Overall Contribution Summary

All work provided, including all queries, database design, and code authored, was done by the only and sole contributor of this project, Wyatt Keller. Approximate hours spent on this portion is ~20 hours.

AI Disclosure Statement

I will only use AI tools for the described use cases, like frontend code, debugging, and UI help. Furthermore, it will assist in general IT work, like port forwarding, ensuring correct connections, the Flask library, etc.

I will not use AI to design the database schema, determine how to normalize, or to generate the SQL queries without proper validation

Signed: Wyatt Keller